PimpMyRide

Project Scope, Use Cases, Domain Model, Database Model & Development Lifecycle

Team Planning Document

Project alignment

Item	Decision
Application	PimpMyRide - vehicle customization marketplace
Primary platform	Cross-Platform (Android + iOS) / React Native + Expo
Backend / database	Firebase Authentication + Firestore + Firebase Storage
External service	Google Maps / geolocation
Device capabilities	GPS and camera/image upload
Version control	GitHub
Core marketplace idea	Customers publish service requests; verified shops discover them and submit quotations
Second feed purpose	Shops showcase completed work for customers to discover

1. Executive Summary

PimpMyRide is a mobile marketplace connecting vehicle owners with verified vehicle customization businesses. The redesigned concept places a two-sided marketplace feed at the centre of the application. Customers maintain one personal account containing multiple vehicles, publish service requests for a selected vehicle, and receive quotations from registered customization shops. Customers then compare quotations and choose a workshop. At the same time, workshops can publish showcase posts displaying completed customization work so customers can discover their services and portfolios.

This approach retains the proposal's original vision of verified workshops, quotation comparison, portfolios, bookings and reviews, while giving the application a clearer differentiating interaction: "Post your ride. Let the right shops come to you."

2. Source Alignment

The project brief requires the team to conceptualise, analyse, design, implement, test and document a mobile application. It also requires persistence, database integration, offline functionality, an internet-based service, a device sensor, API/external-service integration, use cases, UML, a relational model, screen flows, CRUD, debugging, profiling, version control and an end-to-end demonstration.

Module requirement	PimpMyRide implementation
Unique and clear need	A dedicated vehicle-customization marketplace instead of relying on general social media/search.
Domain objects	User, Vehicle, Shop, Service, Post, Quote, Booking, Review, Portfolio Image, Notification, Admin.
Persistence	Firebase/Firestore plus local caching where appropriate.
Database integration	Firestore CRUD for vehicles, shops, services, posts, quotes, bookings and reviews.
Offline functionality	Cached garage, previously loaded marketplace content and booking information.
Internet service	Firebase services and Google Maps/geolocation.
Device sensor	GPS and camera/image capture or image selection.
API/external service	Google Maps API.
Use cases	Customer requests, shop quotations, comparison, booking, showcase posts, reviews, admin verification.
End-to-end demonstration	Account → vehicle → service request → shop quote → comparison → booking → review.
Non-functional demonstration	Usability, reliability, security, maintainability, performance and failure handling.

3. Problem Statement

Vehicle owners often struggle to find trustworthy customization shops and compare pricing, workmanship, reviews and available services. Small workshops also have difficulty marketing their work online. Existing general-purpose platforms are not dedicated to the vehicle-customization quotation process.

PimpMyRide addresses this by creating a dedicated marketplace where customers can publish what they want done to their vehicles, while verified workshops can discover relevant requests, submit quotations and showcase their completed work.

4. Project Vision

To create a trusted mobile marketplace where vehicle owners can manage multiple vehicles, discover customization inspiration, publish service requests and compare quotations from verified workshops, while customization businesses can showcase their work and compete for relevant customer opportunities.

5. Target Users

Actor	Purpose
Customer / Vehicle Owner	Owns one account with multiple vehicles, creates service requests, receives and compares quotes, books workshops and reviews completed work.
Custom Shop	Creates a business profile, lists services, discovers customer requests, submits quotes, manages bookings and showcases completed work.
Administrator	Verifies workshops, moderates marketplace content and manages users/basic analytics.

6. Project Scope

6.1 In Scope - Core MVP

- Customer registration, login, logout and profile management.
- One customer account supporting multiple vehicles.
- Vehicle CRUD: create, view, update and delete vehicles.
- Customer garage showing all vehicles linked to the account.
- Customer service-request posts linked to a selected vehicle.
- Vehicle images attached to service requests.
- Marketplace feed containing customer service requests and workshop showcase posts.
- Workshop registration/profile and verification status.
- Workshop service listings.
- Workshop portfolio/showcase posts, including before-and-after images where appropriate.
- Verified shops discovering customer requests.
- Shops submitting quotations against customer requests.
- Customers receiving and comparing quotations.
- Customer selection of a quotation and appointment booking.
- Booking status management.
- Customer reviews and ratings after completed work.
- Google Maps/location functionality.
- Camera/image upload or image selection.
- Firebase Authentication, Firestore and Firebase Storage.
- Local persistence/caching for selected offline functionality.
- Basic admin verification and moderation.
- GitHub version control and collaborative development.

6.2 Optional / Time-Permitting Features

- Customer-shop chat.
- Notifications beyond essential quote/booking status updates.
- Saved/favourite shops and showcase posts.
- Basic workshop analytics dashboard.
- More advanced search/filtering.
- Project progress updates such as Pending → Confirmed → In Progress → Completed.

6.3 Out of Scope

- AI recommendations.
- AR vehicle visualisation.
- Online payments.
- Loyalty rewards.
- Spare-parts marketplace.
- Insurance integration.
- Full accounting or invoicing system.
- Complex real-time vehicle tracking.
- Large-scale social-media functionality such as stories, reels or follower systems.

7. Core User Journeys

7.1 Primary Customer-to-Shop Marketplace Journey

1. Customer creates or logs into a personal account.
2. Customer adds one or more vehicles to the Garage.
3. Customer selects the vehicle requiring work.
4. Customer creates a service request describing the required customization.
5. Customer uploads relevant vehicle images and optionally adds location, preferred date and budget information.
6. Customer publishes the request to the marketplace feed.
7. Verified shops discover the request.
8. Interested shops submit quotations with price, estimated completion time and message.
9. Customer receives quotation notifications.
10. Customer compares quotations using price, rating, completion time and workshop information.
11. Customer selects the preferred quotation.
12. A booking is created.
13. Workshop manages the booking and updates its status.
14. After completion, customer submits a rating/review.

7.2 Workshop Showcase Journey

15. Workshop logs into its business account.
16. Workshop creates a showcase post.
17. Workshop uploads project images and describes the services performed.
18. The post appears in the marketplace feed.
19. Customers discover the work.
20. Customers can open the workshop profile and view its services, portfolio and reviews.

21. Customers may create a service request and select that workshop directly, if desired.

7.3 Two Ways to Request a Service

Flow	How it works
Open Marketplace Request	Customer publishes a request and allows relevant verified shops to respond with quotations.
Direct Workshop Request	Customer discovers a workshop and sends a service/quote request directly to that workshop.

Both flows should exist because a customer may either know what service is needed but not know which shop to choose, or already have a preferred workshop.

8. Use Cases

8.1 Customer Use Cases

- Register account
- Login
- Manage profile
- Add vehicle
- View vehicle
- Edit vehicle
- Delete vehicle
- Browse marketplace feed
- View showcase post
- View workshop profile
- View workshop services
- Create service request
- Upload vehicle images
- Publish service request
- Edit/cancel service request
- View quotations
- Compare quotations
- Accept/select quotation
- Create booking
- View booking
- Cancel booking where allowed
- View booking status
- Submit review and rating
- View notifications
- Use selected cached features offline

8.2 Custom Shop Use Cases

- Register business account
- Login

- Manage workshop profile
- Submit verification information
- Create/edit/delete services
- Create showcase post
- Upload project images
- Browse customer service requests
- View request and vehicle information
- Submit quotation
- Edit/withdraw quotation where allowed
- Manage bookings
- Update booking status
- View customer reviews
- Respond to/manage business activity

8.3 Administrator Use Cases

- Login as administrator
- View pending workshop registrations
- Verify/reject workshop
- Manage users
- Moderate reported marketplace posts
- Manage inappropriate reviews/content
- View basic platform analytics

8.4 Use Case Diagram - Recommended Structure

For the UML/use-case diagram, place Customer, Custom Shop and Administrator as the three primary actors. Customer should connect to vehicle management, feed browsing, service requests, quotation comparison, bookings and reviews. Custom Shop should connect to profile/services, showcase posts, request discovery, quotations and bookings. Administrator should connect to verification, moderation and analytics.

9. Domain Model

User, Vehicle, Custom Shop, Service, Quote, Booking, Review, PortfolioImage, Notification and Admin as domain objects. The redesigned concept adds Post and explicitly models the relationship between one customer and multiple vehicles.

Class / Object	Key attributes	Main responsibility
User	userId, name, email, phone, role	Stores customer/shop account identity.
Vehicle	vehicleId, ownerId, make, model, year, colour, registration, images	Represents a vehicle owned by a customer.
CustomShop	shopId, ownerId, name, description, location, verified, rating	Represents a customization business.
Service	serviceId, shopId, name, description, estimatedPrice	Defines a service offered by a shop.
Post	postId, authorId, postType, title, description, vehicleId, images, location, status, createdAt	Represents either a customer service request or a shop showcase post.

Quote	quoteId, requestPostId, shopId, price, completionTime, message, status	Represents a shop quotation for a customer request.
Booking	bookingId, customerId, shopId, vehicleId, quoteId, date, time, status	Represents an agreed appointment/service.
Review	reviewId, customerId, shopId, bookingId, rating, comment	Stores customer feedback after completed work.
PortfolioImage	imageId, shopId, postId, imageUrl, caption, type	Stores showcase/portfolio imagery.
Notification	notificationId, userId, type, message, read, createdAt	Alerts users about relevant activity.
Admin	adminId, userId, permissions	Represents administrative privileges.

9.1 Key Relationships

- One User can own many Vehicles.
- One User can create many customer service-request Posts.
- One CustomShop can create many showcase Posts.
- One CustomShop can offer many Services.
- One customer Service Request can receive many Quotes.
- One CustomShop can submit many Quotes.
- One accepted Quote can result in one Booking.
- One Vehicle can be associated with many Service Requests and Bookings over time.
- One Booking can produce one customer Review after completion.
- One CustomShop can have many PortfolioImages/showcase images.
- One User can receive many Notifications.
- An Admin verifies/manages CustomShop accounts and moderates marketplace activity.

9.2 Important Modelling Decision

The customer account is NOT tied to a single vehicle. The relationship is one-to-many: one customer can have multiple vehicles. A service request, quote and booking reference the specific vehicle involved in that transaction. This allows the same customer to request different services for different cars without creating separate accounts.

10. Database Model

Firebase Authentication, Firestore CRUD operations and Firebase Storage. Evidence of data storage and CRUD is required. The following logical model can be used as the design baseline:

Table / Collection	Primary key	Important foreign keys / links
Users	userId	role
Vehicles	vehicleId	ownerId → Users
CustomShops	shopId	ownerId → Users
Services	serviceId	shopId → CustomShops
Posts	postId	authorId → Users; vehicleId → Vehicles when postType = ServiceRequest
Quotes	quoteId	requestPostId → Posts; shopId → CustomShops
Bookings	bookingId	customerId → Users; shopId → CustomShops; vehicleId → Vehicles; quoteId → Quotes

Reviews	reviewId	customerId → Users; shopId → CustomShops; bookingId → Bookings
PortfolioImages	imageId	shopId → CustomShops; postId → Posts where applicable
Notifications	notificationId	userId → Users
Admins	adminId	userId → Users

10.1 Posts: Two Types, One Marketplace Feed

The Posts entity is central to the redesigned concept. A postType field distinguishes the two major feed objects:

postType	Author	Purpose	Important data
SERVICE_REQUEST	Customer	Publishes a vehicle customization need so verified shops can respond.	vehicleId, serviceRequired, description, images, location, budget, preferredDate, status
SHOWCASE	Custom Shop	Displays completed work and attracts potential customers.	servicePerformed, description, images, vehicleType, project details

10.2 Quote Lifecycle

Recommended quotation status flow:

Submitted → Viewed → Accepted / Declined / Withdrawn

Once a quote is accepted, the system can create a Booking.

10.3 Booking Lifecycle

Recommended booking status flow:

Pending → Confirmed → In Progress → Completed

Alternative outcome: Pending → Cancelled.

11. System Architecture

A simple cross-platform architecture. React Native/Expo forms the mobile presentation and application layer. Firebase Authentication handles identity, Firestore stores application data, Firebase Storage handles images, and Google Maps provides mapping/location capabilities. AsyncStorage or an equivalent local mechanism can support selected offline data.

Layer	Technology / component	Purpose
Presentation	React Native + Expo	Mobile screens, forms, navigation and UI.
Navigation	React Navigation	Movement between authentication, customer, shop and admin screens.
Authentication	Firebase Authentication	Registration, login and user identity.
Database	Cloud Firestore	Persistent application data and CRUD.
Storage	Firebase Storage	Vehicle, portfolio and post images.
External service	Google Maps API	Map display and location.

Device capability	GPS / Camera	Location and vehicle/project image capture.
Local persistence	AsyncStorage	Selected cached data and offline support.
Version control	GitHub	Source control, collaboration and evidence.

12. Proposed Screen Structure

Area	Screens
Authentication	Splash, Login, Register, Forgot Password
Customer	Home/Feed, Explore, My Garage, Vehicle Details, Create Request, My Requests, Quotes, Compare Quotes, Bookings, Booking Details, Reviews, Profile, Notifications
Shop	Shop Dashboard, Profile, Services, Create Showcase, Quote Requests, Request Details, Submit Quote, Bookings, Reviews
Admin	Dashboard, Workshop Verification, User Management, Content Moderation, Analytics

13. Project Development Lifecycle

Lightweight Agile approach with short development sprints. Provides well-managed development process covering conceptualisation, analysis, design, implementation, testing, documentation and project management.

Phase	Main activities	Outputs
1. Planning & Requirements	Problem, target users, scope, MVP, requirements, roles, risks, schedule.	Approved scope, requirements, backlog, team roles.
2. Analysis & Design	Use cases, UML, database/relational model, architecture, user flows.	Use-case diagram, class diagram, database model, architecture.
3. UI/UX Design	Figma wireframes, screen flows, design system, prototype.	Approved UI/UX and navigation flow.
4. Implementation	React Native, Firebase, CRUD, marketplace, quotes, bookings, integrations.	Working increments of the application.
5. Testing & QA	Functional, usability, offline, failure, security and performance tests.	Test evidence, bug fixes, profiler/debugging evidence.
6. Integration & Stabilisation	End-to-end flow, physical-device testing, integration fixes.	Stable MVP.
7. Documentation & Presentation	Design document, screenshots, GitHub evidence, presentation and demonstration.	Final documentation, presentation and demo.

13.1 Recommended Sprint Order

22. Sprint 0 — Repository, Expo/React Native setup, Firebase setup, architecture, Figma (interactive prototype screens) and navigation.
23. Sprint 1 — Authentication and customer/shop roles.
24. Sprint 2 — Customer account and multiple-vehicle Garage CRUD.
25. Sprint 3 — Workshop profiles, services and verification.
26. Sprint 4 — Marketplace feed: customer service requests + shop showcase posts.

27. Sprint 5 — Quotation workflow and quote comparison.
28. Sprint 6 — Booking and review workflow.
29. Sprint 7 — Google Maps, GPS, camera/image upload and offline caching.
30. Sprint 8 — Testing, debugging, profiling, physical-device testing and final integration.

14. Team Roles and Responsibilities

Role	Primary responsibility	Key deliverables
1. Project Manager / Business Analyst	Scope, requirements, use cases, planning, coordination and documentation.	Scope, requirements, backlog, use cases, meeting/progress records.
2. UI/UX + Frontend Lead	Figma, navigation, reusable React Native components and customer/shop screens.	Prototype, UI system, screens and navigation.
3. Firebase / Database Lead	Authentication, Firestore, Storage, CRUD, data structure and security rules.	Database, auth, storage, CRUD and data model.
4. Marketplace / Feed Lead	Service-request posts, showcase posts, feed, search/filtering and workshop discovery.	Marketplace feed and post functionality.
5. Quote / Booking / Integration Lead	Quotation workflow, comparison, bookings, reviews, maps, GPS and image integration.	Quotes, booking, reviews and integrations.
QA / DevOps / Documentation Lead (if 6 members)	Testing, GitHub workflow, debugging/profiling evidence and final documentation.	Test plan, GitHub management, QA evidence and final submission.

15. Functional Requirements

ID	Requirement
FR01	The system shall allow customers to register and authenticate.
FR02	The system shall allow a customer account to contain multiple vehicles.
FR03	Customers shall be able to create, retrieve, update and delete vehicles.
FR04	Customers shall be able to browse the marketplace feed.
FR05	Customers shall be able to create service-request posts linked to a selected vehicle.
FR06	Customers shall be able to upload vehicle images to a service request.
FR07	Verified shops shall be able to view relevant customer service requests.
FR08	Shops shall be able to submit quotations against service requests.
FR09	Customers shall be able to view multiple quotations.
FR10	Customers shall be able to compare quotations.
FR11	Customers shall be able to select a quotation.
FR12	The system shall allow a selected quotation to result in a booking.
FR13	Shops shall be able to manage booking status.
FR14	Customers shall be able to submit reviews and ratings for completed work.
FR15	Shops shall be able to create showcase posts.
FR16	Shops shall be able to upload portfolio/showcase images.
FR17	The system shall support workshop verification by administrators.
FR18	The application shall integrate Google Maps/location functionality.
FR19	The application shall use a device capability such as GPS and/or camera.
FR20	The application shall provide selected offline/cached functionality.

16. Non-Functional Requirements

- Usability: Interfaces should be intuitive, consistent and suitable for mobile users.
- Performance: Common screens and cached information should load responsively under normal conditions.
- Security: Authentication and Firebase rules should restrict unauthorised access to protected data.
- Reliability: Network failure should not cause the application to crash.
- Maintainability: The application should use modular components and organised service layers.
- Scalability: The architecture should support additional users, shops, vehicles, posts and transactions.
- Compatibility: The app should operate on recent Android devices and accommodate different screen sizes.
- Availability: Selected previously loaded data should remain accessible when offline.
- Modifiability: Features should be structured so future enhancements can be added without redesigning the entire application.

17. Testing and Evidence Plan

Test area	Evidence to capture
Authentication	Registration/login screenshots and test results.
Vehicle CRUD	Create, retrieve, update and delete evidence in app and database.
Marketplace	Customer request and shop showcase feed screenshots.
Quotation	Shop submitting quote and customer comparison screen.
Booking	Booking creation and status updates.
Reviews	Completed booking followed by rating/review.
Maps/GPS	Map screen and location functionality.
Camera/Image	Image capture/selection and Firebase Storage result.
Offline	App/data behaviour with network disabled.
Debugging	Breakpoint, variable inspection and stepping evidence.
Profiling	CPU/memory or relevant profiler screenshot.
Lifecycle	Evidence of app lifecycle/logging as required by the brief.
GitHub	Repository, branches/commits and collaborative workflow.
Physical device	End-to-end demonstration on a mobile device.

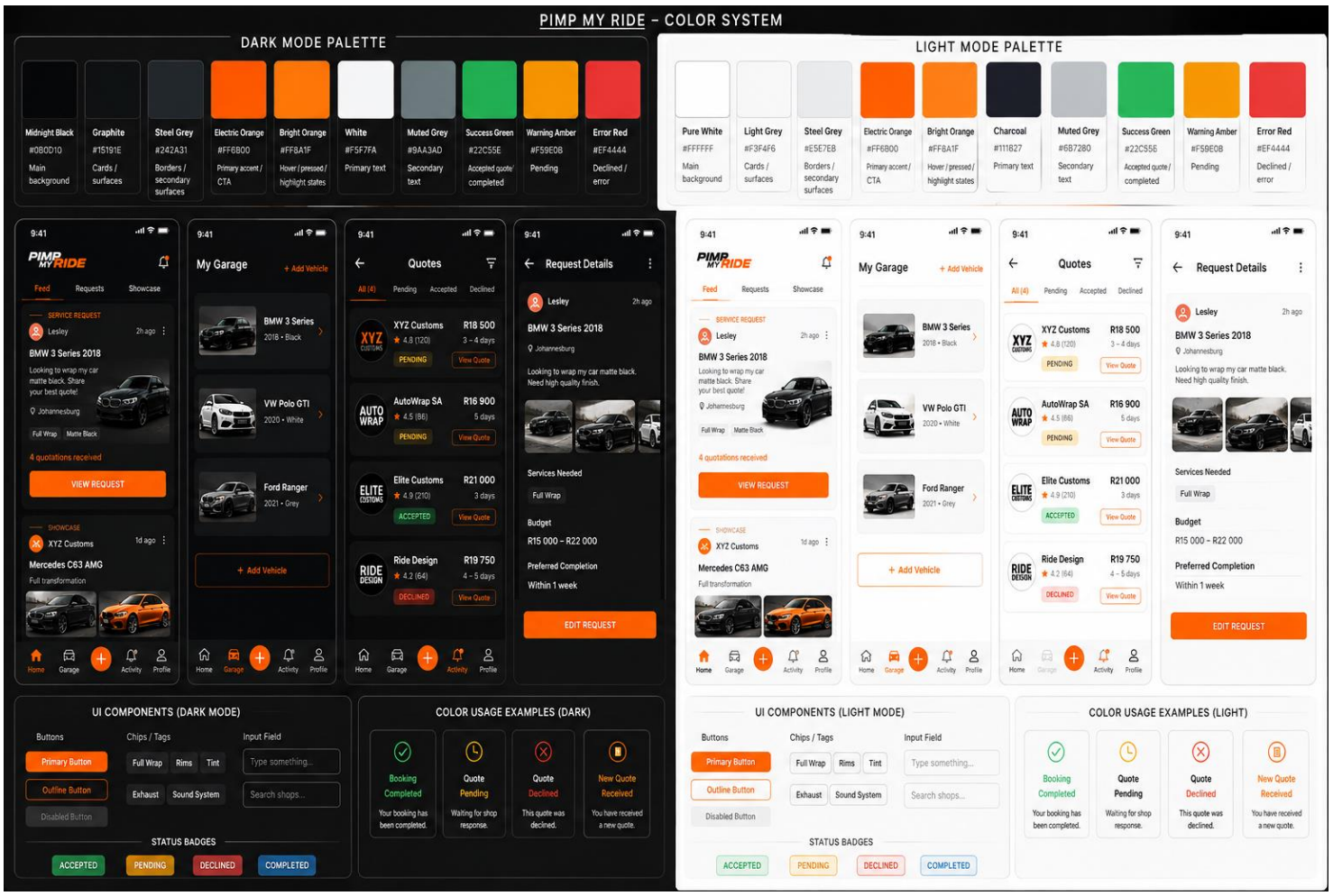
18. Definition of Done

- The UI is implemented.
- Required Firebase/data operations work.
- Validation and basic error handling work.
- The feature has been tested.
- The feature works on the target Android environment.
- The feature is committed to GitHub.
- Another team member has reviewed the implementation.
- Evidence/screenshots required for the project documentation have been captured.

19. Color Scheme

This is the chosen color scheme for both dark and light mode, and we're open for different suggestions.

PLEASE NOTE THAT THE BELOW SCREENSHOT ISN'T OUR SCREEN WIREFRAMES BUT AN AI-GENERATED INSPO.



20. Final Project Pitch

PimpMyRide is a mobile marketplace where vehicle owners manage multiple cars under one account, publish what they want done to a specific vehicle, and receive quotations from verified customization

shops. Customers can compare quotations and book the workshop that best suits their needs, while shops can showcase completed work to attract new customers.

The central idea is simple: instead of forcing customers to search everywhere for the right customization shop, PimpMyRide lets them put their vehicle and requirements in front of relevant businesses and allow the marketplace to bring the opportunities and quotations to them.

END OF TEAM PLANNING DOCUMENT